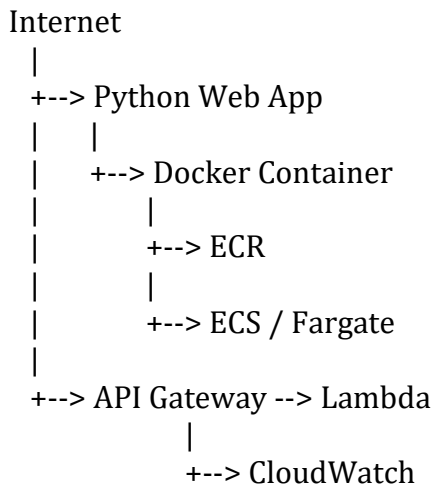


AWS Python Web App — Complete Practical Project Documentation

Python Flask web application using Docker, Amazon ECR, Amazon ECS Fargate, Amazon EC2, AWS Lambda, API Gateway, IAM, and Amazon CloudWatch.

Project Architecture



Project Objectives

Build a Python Flask application, containerize it with Docker, store the image in Amazon ECR, deploy it to EC2 and ECS/Fargate, create a Lambda function, expose Lambda through API Gateway, and monitor execution through CloudWatch.

Technology Stack

Component	Technology
Application	Python + Flask
Container	Docker
Container Registry	Amazon ECR
Compute	Amazon EC2 / ECS Fargate
Serverless	AWS Lambda
API	Amazon API Gateway
Monitoring	Amazon CloudWatch
Access Control	AWS IAM
Region	ap-south-1 (Mumbai)

1. Flask Application

The Flask application provides the main web page and API endpoints. The application listens on 0.0.0.0:5000 so that it can receive traffic from Docker, EC2, or ECS.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "AWS Python Web Application"

@app.route("/api/hello")
def hello():
    return {"message": "Hello from Python Flask!"}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Project Structure

```
aws-python-webapp/
├── app.py
├── requirements.txt
├── Dockerfile
├── .gitignore
├── .dockerignore
├── README.md
├── templates/
│   └── index.html
├── aws/
│   ├── lambda/
│   │   └── lambda_function.py
│   ├── ecs/
│   │   └── task-definition.json
│   └── commands/
│       └── deployment-commands.md
├── screenshots/
└── docs/
```

2. Run Locally

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

```
python app.py
```

Open: <http://localhost:5000>

3. Dockerize the Application

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

```
docker build -t aws-python-webapp .
```

```
docker run -d -p 5000:5000 --name python-webapp aws-python-webapp
```

```
docker ps
```

4. Amazon ECR

Create an ECR repository named `aws-python-webapp` in `ap-south-1` and push the Docker image with the latest tag.

```
aws ecr get-login-password --region ap-south-1 |  
docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.ap-  
south-1.amazonaws.com
```

```
docker tag aws-python-webapp:latest \  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest
```

```
docker push \  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest
```

5. Deploy Docker Image to EC2

Launch an Ubuntu EC2 instance, install Docker, authenticate to ECR, pull the image, and run the container on port 5000.

```
aws ecr get-login-password --region ap-south-1 |  
sudo docker login --username AWS --password-stdin \  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com
```

```
sudo docker pull \  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest
```

```
sudo docker run -d -p 5000:5000 --name python-webapp \  
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest
```

6. AWS Lambda

Create a Python 3.12 Lambda function named `PythonWebAppFunction`. The function returns a JSON response indicating a successful execution.

```
def lambda_handler(event, context):  
    return {  
        "statusCode": 200,  
        "headers": {"Content-Type": "application/json"},  
        "body": '{"message":"Hello from AWS Lambda!","'  
                '"application":"Python Web App",'  
                '"status":"success"}'  
    }
```

7. API Gateway

Create an HTTP API and integrate it with `PythonWebAppFunction`. A route such as `GET /hello` can be used to invoke the Lambda function through an API endpoint.

```
Client  
  |  
  v  
API Gateway  
  |  
  | GET /hello  
  v  
Lambda: PythonWebAppFunction  
  |  
  v  
JSON Response
```

8. ECS Fargate

Create an ECS cluster and deploy the ECR image using AWS Fargate. Use the task definition family `python-webapp-task`, container `python-webapp`, container port 5000, CPU 0.5 vCPU, and memory 1 GB.

```
ECS Cluster  
└─ ECS Service  
    └─ Fargate Task  
        └─ python-webapp :5000  
            └─ ECR Image: aws-python-webapp:latest
```

Recommended ALB Setup

```
Internet
  |
  | HTTP :80
  v
Application Load Balancer
  |
  | HTTP :5000
  v
Target Group
  |
  v
ECS Fargate Task
  |
  v
Docker Container :5000
```

For the ALB setup, keep the public ALB listener on port 80 and configure the target group/container traffic on port 5000. The ECS security group should allow TCP 5000 from the ALB security group.

9. CloudWatch Monitoring

Use CloudWatch Logs to inspect Lambda and ECS application logs. For Lambda, the log group is typically `/aws/lambda/PythonWebAppFunction`. A successful invocation produces START, END, and REPORT log events.

```
CloudWatch
├─ Log groups
│   └─ /aws/lambda/PythonWebAppFunction
│       └─ Latest log stream
│           ├── START
│           ├── application logs
│           ├── END
│           └─ REPORT
```

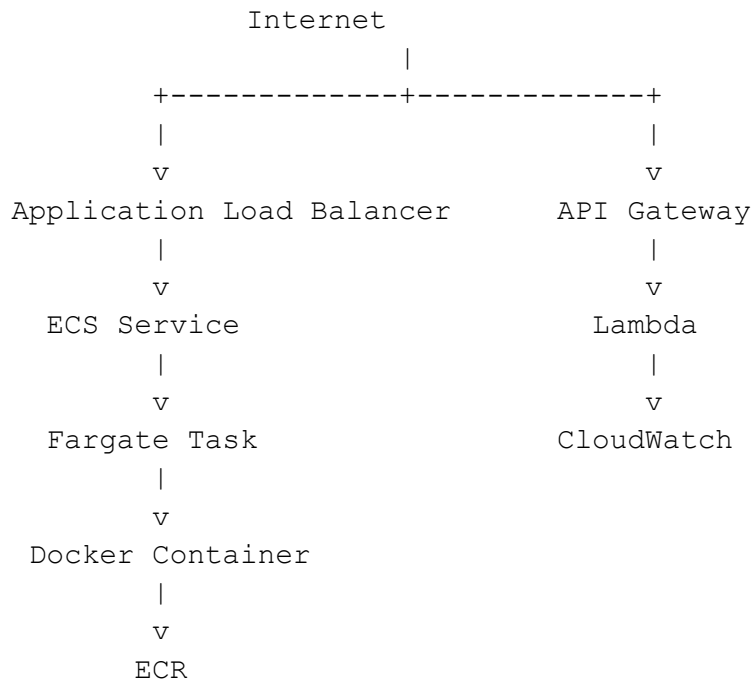
10. GitHub Screenshot Checklist

Filename	What it demonstrates
01-local-application.png	Flask application running locally
02-docker-container.png	Docker image/container running
03-ecr-repository.png	Docker image stored in ECR
04-ec2-docker.png	Container running on EC2
05-ecr-image-on-ec2.png	ECR image pulled and running on EC2
06-lambda-function.png	Lambda function executes successfully
07-api-gateway.png	API Gateway route integrated with Lambda
08-ecs-cluster.png	ECS cluster created
09-ecs-service.png	ECS service and Fargate task running
10-cloudwatch.png	CloudWatch execution/application logs

11. Security and GitHub Best Practices

Never commit AWS access keys, secret keys, passwords, .pem files, .env files, or other credentials. Use placeholders such as <ACCOUNT_ID>, <ECR_URI>, and <EC2_PUBLIC_IP> in documentation. Use IAM roles for AWS services where possible.

12. Final Architecture



EC2 can also pull the same ECR image.

Learning Outcomes

This project demonstrates practical experience with Flask application development, Docker containerization, Amazon ECR image management, EC2 deployment, ECS/Fargate orchestration, Lambda and API Gateway serverless integration, IAM permissions, load balancing concepts, and CloudWatch monitoring.